

Introduction à l'apprentissage profond
TP2
S.Boudra & I.Yahiaoui

Implémentation d'un perceptron et d'un MLP en pur Python

Dans cette partie on va voir de plus près l'implémentation d'un perceptron simple et d'un perceptron multicouche avec du code python pur sans l'utilisation de bibliothèques externes comme pandas, SciKeras et keras.

Perceptron simple

1. Dans un perceptron simple, la sortie est activée lorsque la somme pondérée des entrées dépasse un seuil. On définit une fonction de prédiction pour le calcul de la valeur de sortie prédite :

```
# Make a prediction with weights
def predict(row, weights):
    activation = weights[0]
    for i in range(len(row)-1):
        activation += weights[i + 1] * row[i]
    return 1.0 if activation >= 0.0 else 0.0
```

2. Entraîner le perceptron : apprentissage des poids avec le calcul d'un gradient stochastique. Pour cela, on a besoin de définir deux paramètres : (a) Taux d'apprentissage (b) le nombre d'Epoch.

```
# Estimate Perceptron weights using stochastic gradient descent
def train_weights(train, l_rate, n_epoch):
    weights = [0.0 for i in range(len(train[0]))]
    for epoch in range(n_epoch):
        for row in train:
            prediction = predict(row, weights)
            error = row[-1] - prediction
            weights[0] = weights[0] + l_rate * error
            for i in range(len(row)-1):
                weights[i + 1] = weights[i + 1] + l_rate * error *
row[i]
    return weights
```

3. Application sur la base Sonar¹ qui comporte 208 entrées de 60 variables chacune, et deux (2) classes. Pour cela, on définit les fonctions suivantes :
 - a) La fonction `load_csv` pour charger la base à partir d'un fichier csv.
 - b) La fonction `str_column_to_float` pour convertir les valeurs chargées en string à des valeurs float.
 - c) La fonction `str_column_to_int` pour convertir les classes, dénotées par 'M' et 'R' à des valeurs entières 0 et 1.
 - d) La fonction `evaluate_algorithm` pour estimer les performances dans un contexte de validation croisée avec k-fold à l'aide d'une métrique d'évaluation : la précision (accuracy).
 - e) La fonction `perceptron` qui calcule la valeur de sortie prédite et met à jour les poids des entrées. Pour cela on réutilise les fonctions `predict` et `train_weights`.

Perceptron multicouches (MLP)

Dans cette partie on implémente un perceptron multicouche sur la base Seeds². Les notions de propagation et rétropropagation seront exposées.

La base Seeds contient 201 observations de 7 variables. C'est un problème de classification multi classes avec 3 classes. Dans cet exemple on définit un perceptron multicouche avec une couche en entrée de 7 neurones, une couche cachée de 5 neurones, ensuite une couche de sortie de 3 neurones.

On procède par :

1. Initialisation aléatoire des poids et biais associés aux neurones de chaque couche avec la fonction `initialize_network`.

```
# Initialize a network
def initialize_network(n_inputs, n_hidden, n_outputs):
    network = list()
    hidden_layer = [{'weights':[random() for i in range(n_inputs + 1)]}
    for i in range(n_hidden)]
    network.append(hidden_layer)
    output_layer = [{'weights':[random() for i in range(n_hidden + 1)]}
    for i in range(n_outputs)]
    network.append(output_layer)
    return network
```

2. La propagation : on calcule pour chaque neurone la somme pondérée des variables d'entrées pour une observation donnée avec la fonction `activate`, et sa fonction d'activation avec la fonction `transfer`. On utilise la fonction Sigmoid.

¹ <https://archive.ics.uci.edu/ml/datasets/Connectionist+Bench+%28Sonar%2C+Mines+vs.+Rocks%29>

² <https://archive.ics.uci.edu/ml/datasets/seeds>

```
# Forward propagate input to a network output
def forward_propagate(network, row):
    inputs = row
    for layer in network:
        new_inputs = []
        for neuron in layer:
            activation = activate(neuron['weights'], inputs)
            neuron['output'] = transfer(activation)
            new_inputs.append(neuron['output'])
        inputs = new_inputs
    return inputs
```

3. Calcul de l'erreur : une fois la sortie prédite calculée, on calcul l'erreur de prédiction pour chaque neurone de la couche de sortie. Ici la fonction `transfer_derivative` calcule la dérivée de la fonction d'activation des neurones de sortie.

```
# Backpropagate error and store in neurons
def backward_propagate_error(network, expected):
    for i in reversed(range(len(network))):
        layer = network[i]
        errors = list()
        if i != len(network)-1:
            for j in range(len(layer)):
                error = 0.0
                for neuron in network[i + 1]:
                    error += (neuron['weights'][j] * neuron['delta'])
                errors.append(error)
        else:
            for j in range(len(layer)):
                neuron = layer[j]
                errors.append(neuron['output'] - expected[j])
        for j in range(len(layer)):
            neuron = layer[j]
            neuron['delta'] = errors[j] *
transfer_derivative(neuron['output'])
```

4. Entraîner le réseau de neurones : c'est-à-dire mettre à jour les poids des neurones du réseau avec le mécanisme de rétropropagation de l'erreur de sortie.

```
# Train a network for a fixed number of epochs
def train_network(network, train, l_rate, n_epoch, n_outputs):
    for epoch in range(n_epoch):
        for row in train:
            outputs = forward_propagate(network, row)
            expected = [0 for i in range(n_outputs)]
            expected[row[-1]] = 1
            backward_propagate_error(network, expected)
            update_weights(network, row, l_rate)
```

5. La fonction `evaluate_algorithm` pour estimer les performances dans un contexte de validation croisée avec k-fold à l'aide d'une métrique d'évaluation : la précision (accuracy).

```
def evaluate_algorithm(dataset, algorithm, n_folds, *args):
    folds = cross_validation_split(dataset, n_folds)
    scores = list()
    for fold in folds:
        train_set = list(folds)
        train_set.remove(fold)
        train_set = sum(train_set, [])
        test_set = list()
        for row in fold:
            row_copy = list(row)
            test_set.append(row_copy)
            row_copy[-1] = None
        predicted = algorithm(train_set, test_set, *args)
        actual = [row[-1] for row in fold]
        accuracy = accuracy_metric(actual, predicted)
        scores.append(accuracy)
    return scores
```

Exercices :

1. Ecrivez en python l'algorithme de perceptron qui encode les fonctions logiques 'NON', 'ET', 'OU'.
2. Ecrivez en python l'algorithme de MLP qui encode la fonction logique de XOR.

Implémentation de MLP avec Keras

Dans cette partie on va voir de plus près l'implémentation d'un réseau de neurones avec Keras.

La définition d'un réseau de neurones en Keras passe par les étapes suivantes :

1. Définition du modèle.
2. Compilation du modèle.
3. Entraînement du modèle.
4. Utilisation du modèle entraîné pour la prédiction des classes des nouvelles observations.

Cet exemple sera exécuté sur la base *diabetes*³ qui contient 768 observations avec 8 attributs. C'est un problème de classification binaire. Dans ce cas notre réseau de neurones aura une couche d'entrée de 8 neurones et une couche de sortie de 1 neurone.

Avant de définir le modèle avec Keras, on prépare la base de données, ç.à.d, la charger à partir d'un fichier csv, récupérer la matrice de données et le vecteur des classes. Ici on définit un 'seed' aléatoire pour reproduire notre expérience.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from sklearn.model_selection import train_test_split
import numpy
# fix random seed for reproducibility
seed = 7
numpy.random.seed(seed)
# load the dataset
dataset = numpy.loadtxt("diabetes.csv", delimiter=",")
# split into input (X) and output (Y) variables
X = dataset[:,0:8]
Y = dataset[:,8]
```

1. Définition du modèle : Un modèle Keras est défini sous forme d'une séquence de couches.

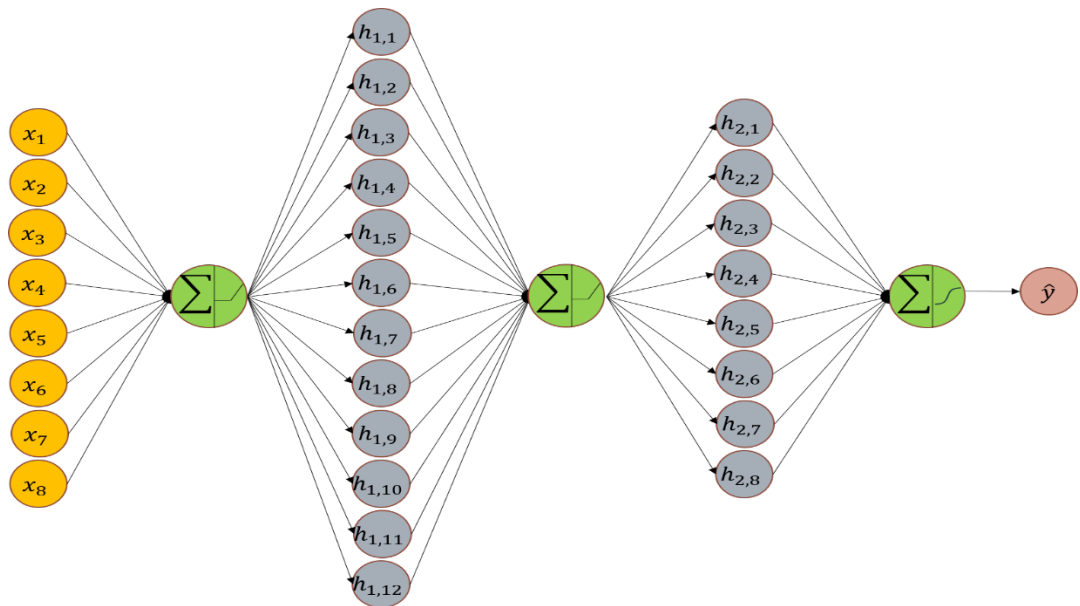
```
# create model
model = Sequential()
model.add(Dense(12, input_dim=8, activation='relu'))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
```

Ici on définit une première couche cachée avec 12 neurones connectés à la couche en entrée de 8 neurones spécifiés par l'attribut `input_dim`. Cela étant, on définit une deuxième couche

³ <https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv>

cachée de 8 neurones, qui, elle-même succède pas une couche de sortie avec un neurone, car il s'agit d'une classification binaire comme le montre la figure ci-dessous.

Pour chaque couche on doit définir une fonction d'activation avec l'attribut `activation`.



2. Compilation du modèle : lors de la compilation du modèle on doit définir les méthodes de calcul de l'erreur de prédiction, la descente de gradient pour la rétropropagation de l'erreur, ainsi que la mesure de performance pour interpréter le score retourné par le réseau de neurones.

```
# Compile model
model.compile(loss='binary_crossentropy', optimizer='adam',
metrics=['accuracy'])
```

Ici, le calcul de l'erreur est exprimé par l'attribut `loss`, dans ce cas une erreur logarithmique. Pour une classification binaire, la valeur `binary_crossentropy` est assigné à `loss`.

`Optimizer` fait référence à la méthode de calcul du gradient pour la rétropropagation de l'erreur. Dans cet exemple, on utilise `adam` qui est une méthode efficace du calcul du gradient stochastique. Pour la mesure de performance, on calcul la précision, i.e., '`accuracy`' qui exprime le ratio entre les observations correctement prédites et le total des observations.

3. Entraîner le modèle : à cette étape on a défini l'architecture du réseau, les paramètres de calcul de l'erreur et le gradient, reste à l'entraîner pour un nombre défini d'epoch avec `epoch` et une taille définie du batch avec `batch_size`. Il s'agit de la mise à jour des poids du réseaux.

```
# Fit the model
model.fit(X,Y, epochs=150, batch_size=10)
```

4. Évaluer le modèle

```
# evaluate the keras model
_, accuracy = model.evaluate(X, Y)
print('Accuracy: %.2f' % (accuracy*100))
```

5. Utiliser le modèle entraîné pour faire des prédictions

```
# make class predictions with the model
predictions = (model.predict(X) > 0.5).astype(int)
# summarize the first 5 cases
for i in range(5):
    print('%s => %d (expected %d)' % (X[i].tolist(), predictions[i],
y[i]))
```

Exercices :

3. Écrivez votre modèle en Keras et testez-le sur la base « diabetes ».
4. Modifiez l'architecture du réseau de neurones en modifiant (a) le nombre de couches cachées. (b) le nombre de neurones dans les couches cachées.

Implémentation de MLP avec SciKeras

On peut implémenter un réseau de neurones avec SciKeras⁴ via les fonctions `KerasClassifier()` pour un problème de classification et `KerasRegressor()` pour un problème de régression linéaire.

Une utilisation typique suit l'exemple suivant :

```
from scikeras.wrappers import KerasClassifier

# define baseline model

def baseline_model():
    # create model
    model = Sequential()
    # Compile model
    return model

estimator = KerasClassifier(model=baseline_model, epochs=200,
batch_size=5, verbose=0)
```

Le modèle Keras est assigné à l'attribut `model` de la fonction `KerasClassifier()`.

⁴ <https://adriangb.com/scikeras/stable/>